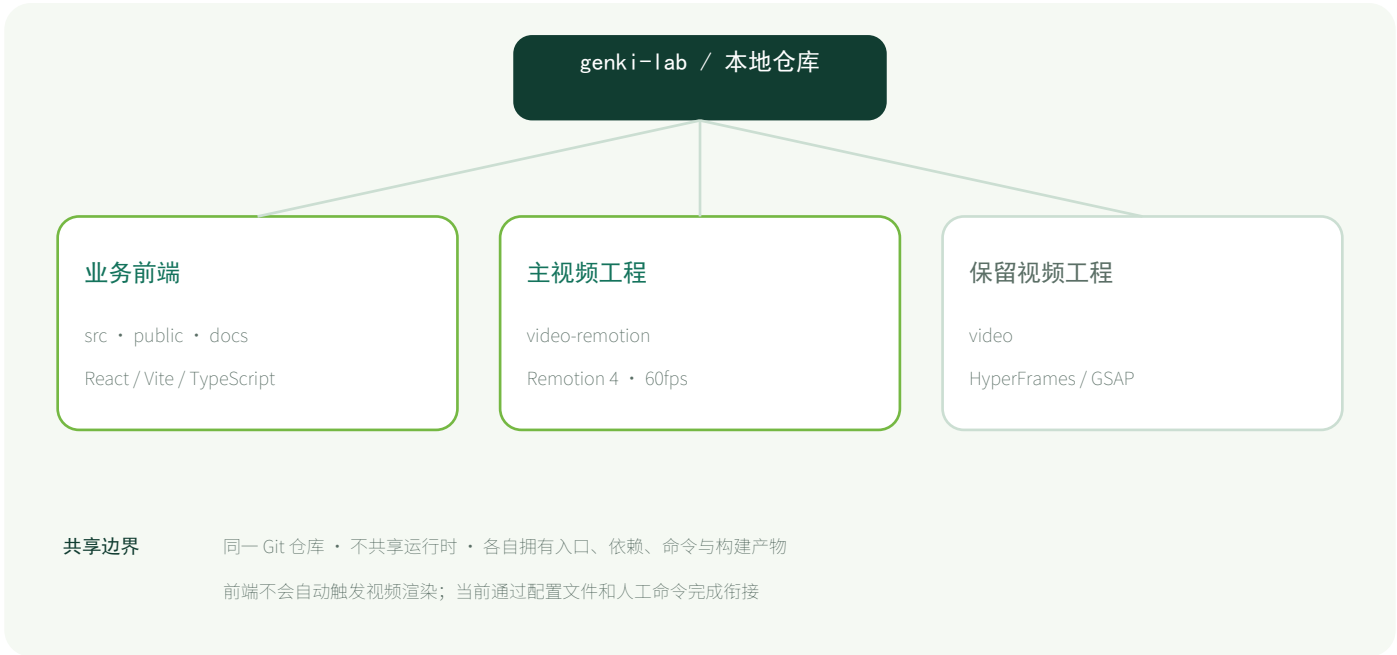


本地仓库架构总览



核心判断

网页系统负责承载创新业务框架，Remotion 负责生成当前 15秒竖屏广告；两者位于同一仓库，但在依赖、运行和构建层面保持隔离。

业务前端 /	video-remotion /	video /
六个路由页面、统一布局、复用组件、类型接口、本地数据层与 GitHub Pages 构建链。	当前主力视频工程。配置驱动、60fps、素材预检、静帧复核、音效与 MP4 渲染。	早期 HyperFrames 方案。独立保留，拥有自己的 JSON、场景代码与依赖，不参与当前成片。

仓库中的四类产物

类型	主要位置	用途	是否直接交付
应用源码	src/	页面、组件、类型、服务与样式	是：构建为 dist/
视频源码	video-remotion/src/	Composition、场景、动效与配置	是：渲染为 MP4
静态素材	public/ 与 video-remotion/public/	产品图、原料图、背景、生成素材与 SFX	随各自工程使用
工程文档	README.md、docs/、video-remotion/docs/	运行、素材、模板、渲染与故障说明	开发交付

仓库规模速览

06 / 页面路由	17 / 复用组件	02 / Composition	01 / Pages 工作流
Dashboard、Research、Trends、Concepts、Content、Validation	布局、状态、业务卡片、图表、内容资产与验证表单	AssetCheck 素材预检 + QingtiJasmineAd 正式成片	main 分支构建根前端并部署 dist，不参与视频渲染

物理目录与工程边界

根目录是业务应用的工程根；两个视频目录分别拥有自己的实现方式。理解这个边界，可以避免在错误目录安装依赖、修改素材或执行渲染命令。

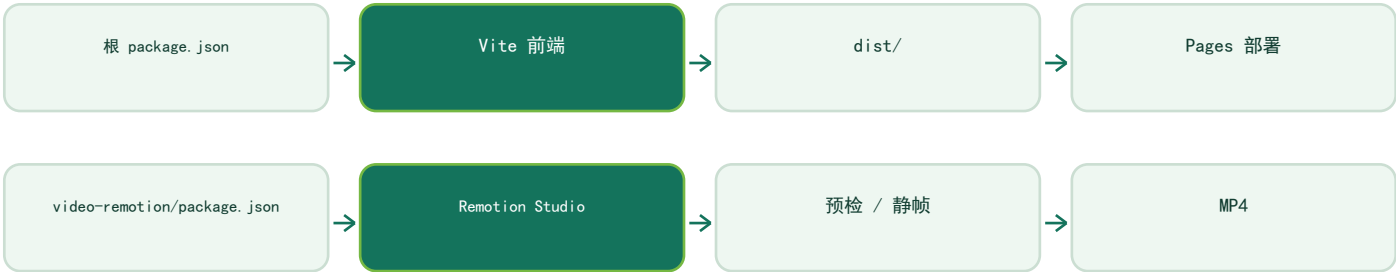
关键目录树

```
genki-lab/
├─ src/ React 业务前端：pages / components / layouts / types / data / services / utils / styles
├─ public/ 前端静态资源与产品素材
├─ docs/ 业务架构与内容规范文档
├─ .github/workflows/deploy-pages.yml 根前端的 GitHub Pages 构建部署
├─ video-remotion/ Remotion 主视频工程：src / public / scripts / docs / out
├─ video/ HyperFrames 保留工程：compositions / data / capture
├─ dist/ Vite 网页构建产物
└─ output/pdf/ 仓库说明类 PDF 交付物
```

三套工程对比

维度	根目录前端	video-remotion	video
核心技术	React + Vite + TypeScript	Remotion 4 + React	HyperFrames + GSAP
入口	src/main.tsx	src/index.ts → Root.tsx	index.html → composition.js
配置	路由、类型与本地数据	ProductVideoConfig	data/qingtj-jasmine.json
主要输出	dist/ 静态站点	out/*.mp4 与 静帧	独立预览/渲染结果
当前定位	业务展示主应用	当前正式视频生产链	历史方案、独立保留

依赖与运行边界



不共享 node_modules 根前端与 Remotion 各自安装依赖，避免 React、TypeScript 与构建工具版本互相污染。	不共享运行时 网页可以单独启动，Remotion 也可以单独预览和渲染；其中一套失败不会阻断另一套。	不自动互调 当前没有后端任务队列，也没有网页按钮直接启动视频渲染。配置与命令是现阶段接口。
--	--	---

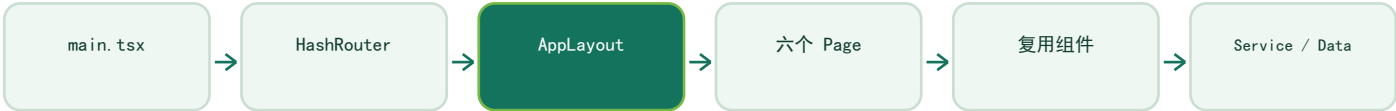
根级关键配置

文件	控制内容	影响范围
package.json	Vite 启动、TypeScript 构建、预览与前端依赖	根业务前端
vite.config.ts	React 插件与相对 base	本地开发、dist、Pages 子路径
tsconfig*.json	应用、Node 配置和项目引用	类型检查与构建
.github/workflows/deploy-pages.yml	Node 22、npm ci、build、Pages 上传部署	main 分支静态站点

业务前端：页面、组件与数据层

根目录前端采用清晰的“入口 → 路由 → 布局 → 页面 → 组件 → 服务 → 本地数据”结构。页面完整但研究内容保持空状态，便于后续接入真实资料。

运行链路



HashRouter 与 Vite base='./' 共同保证静态站点可部署在 GitHub Pages 的子路径下；AppLayout 统一承载桌面导航、移动导航、主内容区和页脚声明。

六个路由页面

路由	页面文件	结构职责
/#/dashboard	DashboardPage.tsx	系统流程图、五个模块入口与阶段提示
/#/research	ResearchPage.tsx	行业报告、竞品、评论、核心洞察四类容器
/#/trends	TrendsPage.tsx	关键词、场景、情绪、健康诉求、机会卡与评分说明
/#/concepts	ConceptsPage.tsx	候选概念、详情、对比、雷达图和状态筛选
/#/content	ContentPage.tsx	产品输入、卖点、文案、海报、脚本、分镜和平台适配
/#/validation	ValidationPage.tsx	用户测试表单与验证结果空面板

组件体系

布局与导航 AppHeader · MobileNavigation · PageTitle	容器与反馈 SectionCard · EmptyState · StatusBadge	业务卡片 ModuleCard · TrendCard · ProductConceptCard · SourceCard
图表与指标 MetricCard · PlaceholderChart · ResultChartCard	内容资产 ContentAssetCard · FlowDiagram	验证输入 ValidationForm：字段已搭建，当前不提交真实数据

数据契约与服务层

TypeScript 接口	对应数据文件	对应服务
ResearchSource / UserComment	researchSources.ts / comments.ts	researchService.ts
TrendOpportunity	trends.ts	trendService.ts
ProductConcept	products.ts	productService.ts
ContentAsset	contentAssets.ts	contentService.ts
ValidationResponse	validationResponses.ts	validationService.ts

data/ 目前保持空数组；Service 只读取本地数据。localStorage 工具使用 genki-lab: 前缀，已经为本地测试预留，但验证表单尚未启用写入。

视觉与响应式落点

global.css 集中维护颜色、渐变、气泡装饰、卡片、网格、按钮、表单与断点规则。	AppHeader 桌面端展示六项导航和 DEMO 标识；移动端把导航切换为折叠菜单。	空状态优先 缺少研究数据时统一使用 EmptyState 与 PlaceholderChart，不用虚构数字填满页面。
---	---	---

主视频工程：Remotion 生产链

video-remotion 是当前广告的唯一主渲染工程。它把产品信息、场景时长、素材路径、视觉组件、声音设计和质量门禁拆开，使广告能够按配置复用。

Composition 与入口

Composition	规格	用途
AssetCheck	1080×1920 · 30fps · 90帧	十项素材与实际字体的竖屏预检画布
QingtiJasmineAd	1080×1920 · 60fps · 900帧 · 15秒	当前青提茉莉气泡茶正式广告

配置驱动路径



产品名称、颜色、口味数组、文案、时长和素材键集中在配置中；FlavorScene 依据 flavors[] 重复生成。新增产品时不需要复制核心场景。

15 秒时间线



场景实现与视觉职责

层级	关键文件 / 组件	职责
场景编排	BeverageProductAd.tsx	按配置计算 Sequence 起止帧与动态口味场景
正式场景	Opening / Reveal / Flavor / Fusion / Bubble / EndCard	从开场、产品揭晓到风味融合、气泡高潮和最终定格
产品主体	ProductBottle	contain 显示、比例保护、轮廓光、阴影、漂浮与运动模糊
环境系统	DynamicBackdrop / RemotionVisualSystem	持续变化的光带、流体、气泡、花瓣、拖影与爆发元素
转场系统	OcclusionTransition	用瓶身、原料、液体或气泡遮挡切点，避免 PPT 式硬切
文字系统	TextReveal / FontGate	中文排版、模糊揭示、安全区与 Chromium 字体门禁
声音系统	SoundDesign	本地 whoosh、impact、pop、sparkle、chime 与画面峰值对齐

60fps 实现原则

设计帧换算	确定性动画	主体清晰度
useDesignFrame() 将 30fps 设计节奏映射到 60fps 输出，时长保持 15 秒。	固定数组、索引与确定性 noise 生成元素位置；不使用真实时间和逐帧随机数。	快速穿梭阶段允许拖影；产品落地后恢复清晰，避免放大低质量包装图。

主视频源码定位

src/config + src/types 产品配置与通用类型 src/compositions/beverage-product-ad Composition、组件与场景
src/assets + src/utls 素材映射与确定性动效工具 scripts + docs 预检、静帧渲染、模板与故障说明

素材、脚本、Skills 与质量门禁

视频工程把“素材能否使用”和“代码能否编译”放在正式渲染之前。所有正式命令都有固定入口，便于重复执行和定位错误。

素材目录与访问规则

位置	内容	访问方式
public/assets/products/qingti-jasmine/	产品正面、主视觉、三视图、青提、茉莉、水花、Logo、源副本	asset-manifest.ts 统一映射
public/assets/generated/qingti-jasmine/	原创建议背景与运动视觉底板	由配置中的资产键引用
public/assets/sfx/	whoosh、impact、pop、sparkle、chime	SoundDesign 按时间线触发
out/	素材检查静帧、七张广告静帧、最终 MP4	渲染命令生成，不手工编辑

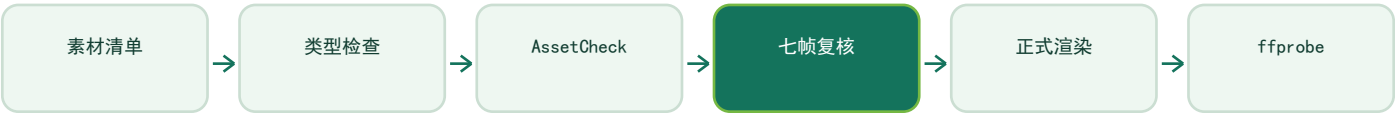
命令链

命令	作用	输出 / 门禁
npm run check:assets	检查十项文件、尺寸、可读性与 Alpha	更新 docs/ASSET_MANIFEST.md
npm run typecheck	执行 tsc --noEmit	阻断类型与配置键错误
npm run dev	启动 Remotion Studio	运行前自动素材检查与类型检查
npm run still:assets	渲染 AssetCheck	out/asset-check.png
npm run stills:ad	输出七张关键正式静帧	检查比例、字体、安全区、转场帧
npm run render:ad	渲染 H.264 成片	out/qingti-jasmine-ad.mp4

项目本地 Skills

remotion-best-practices 官方开发规范：Composition、资源、时序和性能基线。	remotion-create 工程创建、参数化、场景时序和可复用组件约束。	remotion-markup 中文排版、安全区、图像 contain、转场与文字测量。
remotion-render 素材预检、字体门禁、静帧检查、MP4 与 ffprobe 验证。	remotion-beverage-ad-motion 本项目沉淀的 60fps、动态背景、主体分离、遮挡转场和音效经验。	AGENTS.md 视频工程的本地开发约束，限制素材、字体、随机性、包装处理和正式渲染条件。

质量流水线



视频工程文档索引

文档	回答的问题
REMOTION_SETUP.md / RENDER_GUIDE.md	环境如何安装、Studio 如何预览、静帧与 MP4 如何输出
ASSET_MANIFEST.md / CREATIVE_ASSETS.md	有哪些素材、尺寸与 Alpha 是否合格、生成素材如何使用
BATCH_TEMPLATE_GUIDE.md	如何增加产品配置、资产目录和新的 Composition
VIDEO_BRIEF.md / TROUBLESHOOTING.md	成片目标是什么、失败时从素材、字体或渲染哪一层排查

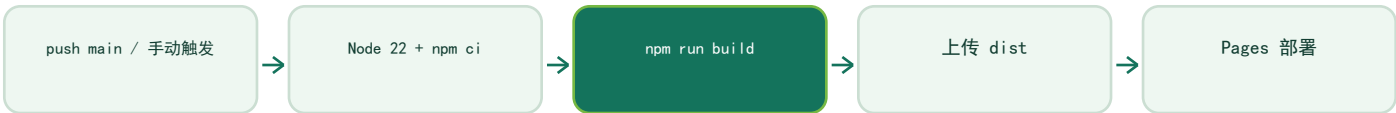
构建交付与新增产品路径

根前端和主视频工程拥有两条独立的交付链。现有 GitHub Actions 只部署网页；视频仍在本地经过素材、静帧和媒体参数检查后输出。

两条交付链

对象	本地命令	构建产物	现有自动化
GENKI LAB 网页	根目录 npm run build	dist/ 静态站点	push main → deploy-pages.yml → GitHub Pages
竖屏产品广告	video-remotion 内 npm run render:ad	out/qingti-jasmine-ad.mp4	当前为本地手动渲染与复核
架构说明文档	tmp/pdfs/ 中生成与渲染校验	output/pdf/*.pdf	当前为本地生成

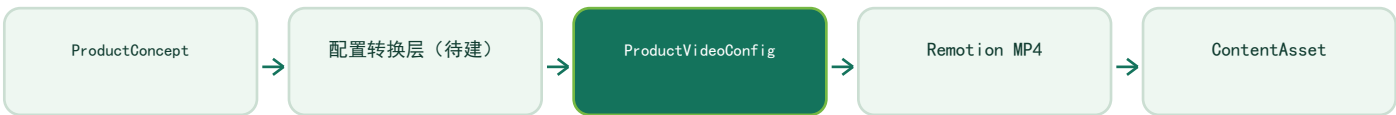
GitHub Pages 工作流的实际范围



新增一个产品广告配置

步骤	修改位置	完成标准
01 素材落盘	public/assets/products// 与 generated//	产品图可读、原料透明、文件名稳定
02 登记路径	src/assets/asset-manifest.ts	只新增资产键，不在场景中散写路径
03 新增配置	src/config/.ts	满足 ProductVideoConfig，填写颜色、文案、口味、时长与 SFX
04 注册成片	src/Root.tsx	增加 Composition，并把配置作为 inputProps
05 预检复核	check:assets → AssetCheck → stills:ad	主体比例、字体、安全区、切点与音效节奏合格
06 正式渲染	remotion render 或对应 npm script	输出 1080×1920 MP4，并验证编码、帧率与时长

当前架构的连接点



现在如何工作	后续最小改造	保持不变的边界
研究与产品页面展示结构；视频工程读取自己的 typed config；两者通过人工整理配置衔接。	增加 ProductConcept → ProductVideoConfig 的转换器，即可减少重复录入，无需重写场景。	网页继续负责业务展示，Remotion 继续负责媒体渲染；素材门禁和静帧复核仍然独立执行。

仓库已经具备完整的前端展示骨架和可复用的视频生产骨架；下一步的重点是让真实产品数据以清晰、可校验的配置进入现有两条链路。